

National Cipher Challenge 2025

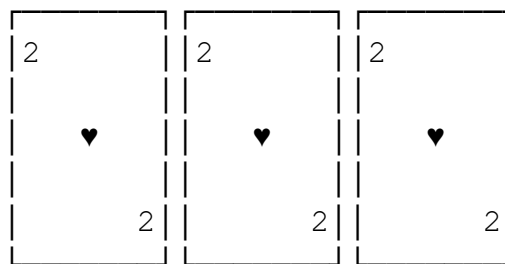
Challenge 10B (final challenge)

Solution by Qubit

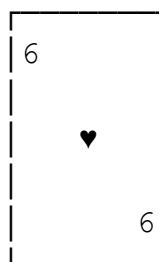
When we opened the challenge page, we saw this ciphertext:

```
7CXS3 H6S7C KSXDA S2CKS 8D4S9 C2S7D XS3D9 S7C2S
6HJS9 C2S3D 7S8CQ HACQS AH5S9 DJH7D 3S8D5 S7CQH
3D8S2 CKSAH 3S7CX S6DJH XCJHX C9S8D ASXCA S8CJH
8D5SA CQHJD 5SJC2 SXCXS XC9SA CJHJC KH2CK S8D4S
8D4S7 C2SQD KS5CJ H2CKS 2D3S9 CQSQC 3S5CJ H2CKS
9C5S6 DXS9D 6S6HK S2H2S 8DKSX CQH7D XSXDA S2CKS
AH3S7 CXS6D JHXCJ HXC9S JC4S9 C2S8C ...
```

Of course we were terrified, but as always we started with frequency analysis and found lots of Cs, which was not very helpful. We also counted the total number of characters: 6644, which is divisible by 2, 4, 11 and 151.



After several minutes of staring at the ciphertext one of us realised that every pair of characters represents a solitaire card. For example **6S** represents:



6 OF SPADES

The first thing that comes to mind after seeing a solitaire card is the Solitaire cipher, so we spent the next few hours looking for links between the ciphertext and Solitaire. But as you know, the Solitaire cipher uses cards to generate a keystream while the ciphertext itself is ordinary alphabet text like any stream cipher. After several hours we concluded the cipher could not be Solitaire, so we started considering other possibilities.

We focused on the ciphertext for clues. Look at the first 12 characters:

7C XS 3H 6S 7C KS

Then we noticed a strange repeat of **C**: clubs appeared again and again with only one card between them, though sometimes hearts appeared instead. We began to analyse the ciphertext for links between suits.

We put the cards in this table which is ordered by the general value of cards and HEART is spited between two squares.

[illegible]

This is our python file for decryption:

```
# decryption.py

import os

# card grid: clubs/diamonds/hearts (6x6)
UL = [
    ["AC", "2C", "3C", "4C", "5C", "6C"],
    ["7C", "8C", "9C", "XC", "JC", "QC"],
    ["KC", "AD", "2D", "3D", "4D", "5D"],
    ["6D", "7D", "8D", "9D", "XD", "JD"],
    ["QD", "KD", "AH", "2H", "3H", "4H"],
    ["5H", "6H", "7H", "8H", "9H", "XH"],
]

# card grid: spades + some hearts (4x4)
LR = [
    ["JH", "QH", "KH", "AS"],
    ["2S", "3S", "4S", "5S"],
    ["6S", "7S", "8S", "9S"],
    ["XS", "JS", "QS", "KS"],
]

# letter grid for lower-left square
LL_LETTERS = [
    ['S', 'H', 'A', 'D', 'O', 'W'],
    ['B', 'C', 'E', 'F', 'G', 'I'],
    ['K', 'L', 'M', 'N', 'P', 'Q'],
    ['R', 'T', 'U', 'V', 'X', 'Y'],
]

# letter grid for upper-right square
UR_LETTERS = [
    ['F', 'U', 'L', 'H'],
    ['E', 'A', 'R', 'T'],
    ['B', 'C', 'D', 'G'],
    ['I', 'K', 'M', 'N'],
    ['O', 'P', 'Q', 'S'],
    ['V', 'W', 'X', 'Y'],
]

# -----
# tokenizer / validation
# -----
# valid rank and suit characters
```

```

_RANKS = set("A23456789XJQK")
_SUITS = set("CDHS")

# simple check for a 2-char card token
def _is_card(t):
    return len(t) == 2 and t[0] in _RANKS and t[1] in _SUITS

def read_cipher_cards(filename: str):
    # prefer given path, else try same file next to the script
    script_dir = os.path.dirname(os.path.abspath(__file__))
    alt = os.path.join(script_dir, filename)

    if os.path.isfile(filename):
        path = filename
    elif os.path.isfile(alt):
        path = alt
    else:
        attempted = [os.path.abspath(filename), os.path.abspath(alt)]
        raise FileNotFoundError(f"cipher text file not found; tried: {attempted}")

    # read and normalise to upper-case
    with open(path, "r", encoding="utf-8") as fh:
        s = fh.read().strip().upper()

    if not s:
        raise ValueError(f"{filename} is empty")

    # if separators present, split on them
    if any(c in s for c in " \t\n;"):
        s = s.replace(",", " ").replace(";", " ")
        tokens = [p for p in s.split() if _is_card(p)]
    else:
        # no separators – slide a 2-char window and pick valid tokens
        tokens = []
        i = 0
        n = len(s)
        while i <= n - 2:
            cand = s[i:i+2]
            if _is_card(cand):
                tokens.append(cand)
                i += 2
            else:
                i += 1

    if not tokens:

```

```

        raise ValueError(f"No valid card tokens found in {filename}")

    # if odd count, drop last token (keep going rather than failing hard)
    if len(tokens) % 2:
        print(f"Warning: odd number of card tokens ({len(tokens)}); dropping
final token")
        tokens = tokens[:-1]

    return tokens

# -----
# four-square helpers
# -----
def find_position(grid, target):
    # return (row, col) for the target, or None
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == target:
                return r, c
    return None

def decrypt(cards):
    # process pairs of cards into plaintext digraphs
    out = []
    for i in range(0, len(cards), 2):
        a, b = cards[i], cards[i+1]
        p1 = find_position(UL, a)
        p2 = find_position(LR, b)
        if p1 is None or p2 is None:
            raise ValueError(f"Unknown card token: {a} or {b}")
        r1, c1 = p1
        r2, c2 = p2
        # map into letter grids (note order: LL letter then UR letter)
        out.append( LL_LETTERS[r2][c1] + UR_LETTERS[r1][c2] )
    return "".join(out)

# -----
# quick CLI entry
# -----
if __name__ == "__main__":
    cards = read_cipher_cards("cipher.txt")
    print(decrypt(cards))

```

At the start, we didn't know how the cards were arranged in the missing squares, and we also didn't know the key grids needed to decrypt the message. Since there are far too many possible arrangements to try by hand, the only practical option was to search for them programmatically.

The first step was to write a basic decryption routine. This function takes a proposed key (the two unknown Four-Square grids), reads the ciphertext from `cipher.txt`, and applies the Four-Square rules using the fixed card squares. Given a key, it produces a candidate plaintext. On its own this doesn't tell us whether the result is meaningful English, but it lets us test keys quickly.

Next, we built the main solver around this decryption function. The solver uses quadgram analysis to evaluate how "English-like" a decrypted text is. Quadgrams are sequences of four letters, and by using frequency data from `english_quadgrams.txt`, we can assign a numerical score to any piece of text. Text with common English letter patterns gets a higher score, while random noise scores poorly. This gives the program an objective way to compare different keys.

The solving process starts by generating random keys for the unknown grids. For each key, the ciphertext is decrypted and scored using the quadgram fitness function. The program then makes small changes to the key, typically by swapping two letters in one of the grids, and decrypts the text again. If the new key produces a higher score, it is kept; if not, it is discarded. By repeating this process thousands of times, the key gradually improves and the plaintext becomes more readable.

To avoid getting stuck in a bad local solution, the program performs multiple restarts. Each restart begins with a fresh random key and runs the same improvement process. At the end, the best-scoring result across all restarts is selected as the final output.

For the solver to work correctly, `english_quadgrams.txt` must be located in the same directory as `four-s.py`, since it is loaded at runtime to build the scoring model. Without this file, the program has no way to judge whether a decrypted message looks like real English.

Our python code:

```
# four-s.py
```

```

import random
import math
import copy
import os

# =====
# STATIC CARD GRIDS
# =====

# 6x6 card square (Clubs, Diamonds, Hearts A..X)
UL = [
    ["AC", "2C", "3C", "4C", "5C", "6C"],
    ["7C", "8C", "9C", "XC", "JC", "QC"],
    ["KC", "AD", "2D", "3D", "4D", "5D"],
    ["6D", "7D", "8D", "9D", "XD", "JD"],
    ["QD", "KD", "AH", "2H", "3H", "4H"],
    ["5H", "6H", "7H", "8H", "9H", "XH"],
]

# 4x4 card square (Spades + face hearts)
LR = [
    ["JH", "QH", "KH", "AS"],
    ["2S", "3S", "4S", "5S"],
    ["6S", "7S", "8S", "9S"],
    ["XS", "JS", "QS", "KS"],
]

# =====
# CARD TOKEN PARSER
# =====

# Simple check for valid card tokens
def is_valid_card(token):
    if len(token) != 2:
        return False
    rank = token[0]
    suit = token[1]
    return rank in "A23456789XJQK" and suit in "CDHS"

def read_cipher_cards(filename: str):
    raw = None

    # Try current working directory first
    if os.path.isfile(filename):
        with open(filename, 'r', encoding='utf-8') as f:
            raw = f.read()

```

```

else:
    # Fall back to script directory
    script_dir = os.path.dirname(os.path.abspath(__file__))
    alt_path = os.path.join(script_dir, filename)
    if os.path.isfile(alt_path):
        with open(alt_path, 'r', encoding='utf-8') as f:
            raw = f.read()
    else:
        raise FileNotFoundError(f"ciphertext file not found: {filename}")

text = raw.strip().upper()
if not text:
    raise ValueError(f"{filename} is empty")

cards = []

# If separators exist, split on them
if any(c in text for c in " ,;\n\t"):
    parts = text.replace(",", " ").replace(";", " ").split()
    for p in parts:
        if is_valid_card(p):
            cards.append(p)
else:
    # Otherwise assume concatenated format
    i = 0
    while i + 1 < len(text):
        token = text[i:i+2]
        if is_valid_card(token):
            cards.append(token)
            i += 2
        else:
            i += 1 # slide forward if something odd shows up

if not cards:
    raise ValueError(f"No valid card tokens found in {filename}")

# Drop the last token if count is odd
if len(cards) % 2 != 0:
    print(f"Warning: odd number of card tokens ({len(cards)}); dropping
final token")
    cards = cards[:-1]

return cards

# =====
# FOUR-SQUARE DECRYPTION

```



```

# =====

def find_position(grid, target):
    # Basic lookup, grids are small so this is fast enough
    for r, row in enumerate(grid):
        for c, val in enumerate(row):
            if val == target:
                return r, c
    return None

def decrypt(cards, LL, UR):
    plaintext = []

    # Process cards as digraphs
    for i in range(0, len(cards), 2):
        c1 = cards[i]
        c2 = cards[i + 1]

        pos1 = find_position(UL, c1)
        pos2 = find_position(LR, c2)

        if pos1 is None or pos2 is None:
            raise ValueError(f"Invalid card in ciphertext: {c1} or {c2}")

        r1, c1 = pos1
        r2, c2 = pos2

        # Four-square swap
        p1 = UR[r1][c2]
        p2 = LL[r2][c1]

        plaintext.append(p2 + p1)

    return ''.join(plaintext)

# =====
# HILL-CLIMBING FUNCTIONS
# =====

def load_quadgrams(filename='english_quadgrams.txt'):
    quadgrams = {}

    if not os.path.isfile(filename):
        raise FileNotFoundError(
            f"Quadgram file '{filename}' not found. "

```

```

        "Download from practicalcryptography.com"
    )

with open(filename, 'r') as f:
    for line in f:
        parts = line.strip().split()
        if len(parts) != 2:
            continue
        quad, count_str = parts
        if len(quad) != 4:
            continue
        try:
            count = int(count_str)
            if count > 0:
                quadgrams[quad] = count
        except ValueError:
            pass

if not quadgrams:
    raise ValueError("No quadgrams loaded from file.")

return quadgrams

def fitness(text, log_probs, floor):
    # Average quadgram log score
    if len(text) < 4:
        return 0.0

    score = 0.0
    for i in range(len(text) - 3):
        quad = text[i:i+4]
        score += log_probs.get(quad, floor)

    return score / (len(text) - 3)

def random_grid(rows, cols, alphabet):
    # Shuffle letters into a grid
    letters = list(alphabet)
    random.shuffle(letters)
    return [letters[i * cols:(i + 1) * cols] for i in range(rows)]

def swap_in_grid(grid):
    # Swap two random positions
    rows = len(grid)
    cols = len(grid[0])

```

```

a = random.randrange(rows * cols)
b = random.randrange(rows * cols)
while a == b:
    b = random.randrange(rows * cols)

r1, c1 = divmod(a, cols)
r2, c2 = divmod(b, cols)

new_grid = copy.deepcopy(grid)
new_grid[r1][c1], new_grid[r2][c2] = new_grid[r2][c2], new_grid[r1][c1]
return new_grid

# =====
# MAIN
# =====

if __name__ == "__main__":
    cards = read_cipher_cards("cipher.txt")
    print(f"Loaded {len(cards)} card tokens from ciphertext.")

    quadgrams = load_quadgrams()
    total = sum(quadgrams.values())
    log_probs = {k: math.log(v / total) for k, v in quadgrams.items()}
    floor = math.log(0.01 / total)
    print("Quadgram statistics loaded.")

    alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'

    best_score = float('-inf')
    best_LL = None
    best_UR = None
    best_text = ""

    num_restarts = 5
    iterations_per_restart = 10000

    for restart in range(num_restarts):
        print(f"\nRestart {restart + 1}/{num_restarts}")

        LL_LETTERS = random_grid(4, 6, alphabet)
        UR_LETTERS = random_grid(6, 4, alphabet)

        current_text = decrypt(cards, LL_LETTERS, UR_LETTERS)
        current_score = fitness(current_text, log_probs, floor)

```

```

for i in range(iterations_per_restart):
    if random.random() < 0.5:
        new_LL = swap_in_grid(LL_LETTERS)
        new_UR = UR_LETTERS
    else:
        new_LL = LL_LETTERS
        new_UR = swap_in_grid(UR_LETTERS)

    new_text = decrypt(cards, new_LL, new_UR)
    new_score = fitness(new_text, log_probs, floor)

    if new_score > current_score:
        LL_LETTERS = new_LL
        UR_LETTERS = new_UR
        current_text = new_text
        current_score = new_score

    if (i + 1) % 1000 == 0:
        print(f"  Iter {i + 1}: score {current_score:.4f} | text:
{current_text[:50]}...")

    if current_score > best_score:
        best_score = current_score
        best_LL = copy.deepcopy(LL_LETTERS)
        best_UR = copy.deepcopy(UR_LETTERS)
        best_text = current_text
        print(f"New best score: {best_score:.4f}")

print("\nHill-climbing complete.")
print("Best plaintext:")
print(best_text)

print("\nBest LL_LETTERS grid:")
for row in best_LL:
    print(row)

print("\nBest UR_LETTERS grid:")
for row in best_UR:
    print(row)

```

WRITTEN BY AMIN BAHRAMI